

MENDSPEECH REVISED LEARNING SYSTEM

Week 2: ASR, CTC, Confidence, and Repair Localization

Build the recognition and uncertainty layer, plus a reusable Modal execution path.

Six focused build days plus one consolidation day. Each day ends with named artifacts or measured evidence.

Week milestone

Build the recognition and uncertainty layer, plus a reusable Modal execution path.

Week map

Day	Focus	Minimum evidence	Compute
Day 8	Frame sequence to transcript	• Compare clean and corrupted transcripts on the exact same utterances.	Modal L4 optional, CPU acceptable for small runs
Day 9	CTC from first principles	• Enumerate several legal paths for a tiny target word. • Break your decoder deliberately with repeated letters and fix it.	Local CPU
Day 10	WER, CER, and error taxonomy	• Score clean versus every SpeechDamageBench severity. • Find which corruption type causes deletion errors fastest.	Local CPU
Day 11	Token confidence and uncertainty	• Compare confidence on clean, noisy, clipped, and dropout audio. • Find confident but wrong examples and document them.	Modal L4 recommended
Day 12	Time alignment and uncertain spans	• Inject known 100 ms and 250 ms dropouts and test whether uncertainty overlaps them.	Modal L4 recommended
Day 13	Define selective repair policy v0	• Sweep confidence thresholds on SpeechDamageBench. • Measure percentage of audio selected for repair and overlap with known damaged spans.	Local CPU after ASR outputs are cached
Day 14	Week 2 integration and review	• Run at least twenty corrupted utterances and manually inspect policy errors.	Modal L4 recommended

Reference spine

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

Frame sequence to transcript

Compute

Modal L4 optional, CPU acceptable for small runs

LEARN

- Why acoustic frames outnumber output tokens.
- Encoder outputs, vocabulary logits, and decoding.
- CTC versus transducer versus attention decoder at a high level.

BUILD IN MENDESPEECH

- Run a pretrained ASR model on clean and damaged SpeechDamageBench clips.
- Store transcript, token outputs if available, and timing metadata.
- Add a reusable Modal entry point so the same command can run ASR experiments on an L4 without editing deployment code each day.

EXPERIMENT AND MEASURE

- Compare clean and corrupted transcripts on the exact same utterances.

REQUIRED OUTPUT

- `src/asr/baseline.py`
- `infra/modal_asr.py`
- `results/day08_baseline_transcripts.csv`

Completion check: You can draw the path from features to encoder states to token probabilities to text, and launch the same baseline locally or on Modal with a documented command.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

CTC from first principles

LEARN

- CTC blank symbol.
- Repeated labels and collapse operation.
- Why many frame paths map to one transcript.
- Conditional independence assumption and its consequence.

BUILD IN MENDESPREECH

- Implement CTC collapse yourself without a library decoder.
- Create hand written alignment examples and unit tests.

EXPERIMENT AND MEASURE

- Enumerate several legal paths for a tiny target word.
- Break your decoder deliberately with repeated letters and fix it.

REQUIRED OUTPUT

- `src/asr/ctc_decode.py`
- `tests/test_ctc_decode.py`
- `docs/ctc_explained.md`

Completion check: You can explain why a blank is needed and correctly decode repeated characters.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

WER, CER, and error taxonomy

LEARN

- Word error rate: substitutions, deletions, insertions.
- Character error rate and when it helps.
- Why WER alone hides error severity.

BUILD IN MENDESPREECH

- Implement or verify WER and CER calculations.
- Add an error analyzer that labels substitution, deletion, and insertion spans.

EXPERIMENT AND MEASURE

- Score clean versus every SpeechDamageBench severity.
- Find which corruption type causes deletion errors fastest.

REQUIRED OUTPUT

- src/metrics/wer.py
- results/day10_wer_by_damage.csv
- results/day10_error_types.csv

Completion check: You can calculate WER by hand for a short example and explain each error.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

Token confidence and uncertainty

LEARN

- Softmax confidence and why it can be miscalibrated.
- Frame confidence versus token confidence versus word confidence.
- Entropy as an uncertainty signal.
- Confidence calibration intuition.

BUILD IN MENDSPEECH

- Extract confidence or approximate it from model outputs.
- Create a word level confidence timeline aligned to the transcript.

EXPERIMENT AND MEASURE

- Compare confidence on clean, noisy, clipped, and dropout audio.
- Find confident but wrong examples and document them.

REQUIRED OUTPUT

- `src/asr/confidence.py`
- `results/day11_confidence_cases.csv`
- `docs/confidence_failure_modes.md`

Completion check: You understand why low confidence can be useful but cannot be treated as truth.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

Time alignment and uncertain spans

LEARN

- Frame time conversion.
- Token timestamps and word timestamps.
- Alignment boundaries around corrupted regions.

BUILD IN MENDESPEECH

- Map low confidence tokens back to audio time spans.
- Overlay uncertain intervals on waveform and spectrogram.

EXPERIMENT AND MEASURE

- Inject known 100 ms and 250 ms dropouts and test whether uncertainty overlaps them.

REQUIRED OUTPUT

- `src/asr/alignment.py`
- `app/uncertainty_overlay.py`
- `results/day12_overlap_metrics.csv`

Completion check: The UI can highlight an uncertain audio interval and show the associated word or token.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

Define selective repair policy v0

Compute

Local CPU after ASR outputs are cached

LEARN

- Threshold policies.
- Hysteresis to avoid rapid toggling.
- Minimum repair span and padding.
- False repair versus missed repair tradeoff.

BUILD IN MENDSPEECH

- Implement Preserve, Balanced, and Rescue policies.
- Each policy returns preserve spans and repair spans from confidence plus timing.

EXPERIMENT AND MEASURE

- Sweep confidence thresholds on SpeechDamageBench.
- Measure percentage of audio selected for repair and overlap with known damaged spans.

REQUIRED OUTPUT

- src/controller/policy.py
- configs/repair_modes.yaml
- results/day13_policy_sweep.csv

Completion check: You can explain the cost of repairing too much and repairing too little.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence

Week 2 integration and review

LEARN

- Review CTC, WER, confidence, timestamp alignment, and repair decisions.

BUILD IN MENDSPEECH

- Pipeline: damaged audio to transcript to confidence to highlighted repair spans.
- Add clean JSON output for every run.
- Verify the Modal wrapper records model revision, GPU type, software versions, and run id automatically.

EXPERIMENT AND MEASURE

- Run at least twenty corrupted utterances and manually inspect policy errors.

REQUIRED OUTPUT

- app/mendspeech_v0.py
- infra/modal_asr.py
- results/week2_casebook.md
- reports/week2_asr_uncertainty.md

Completion check: A user can see what the ASR heard and which exact intervals MendSpeech wants to preserve or repair.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- CTC primary paper or a reliable derivation
- Framework ASR documentation for logits, timestamps, and confidence